

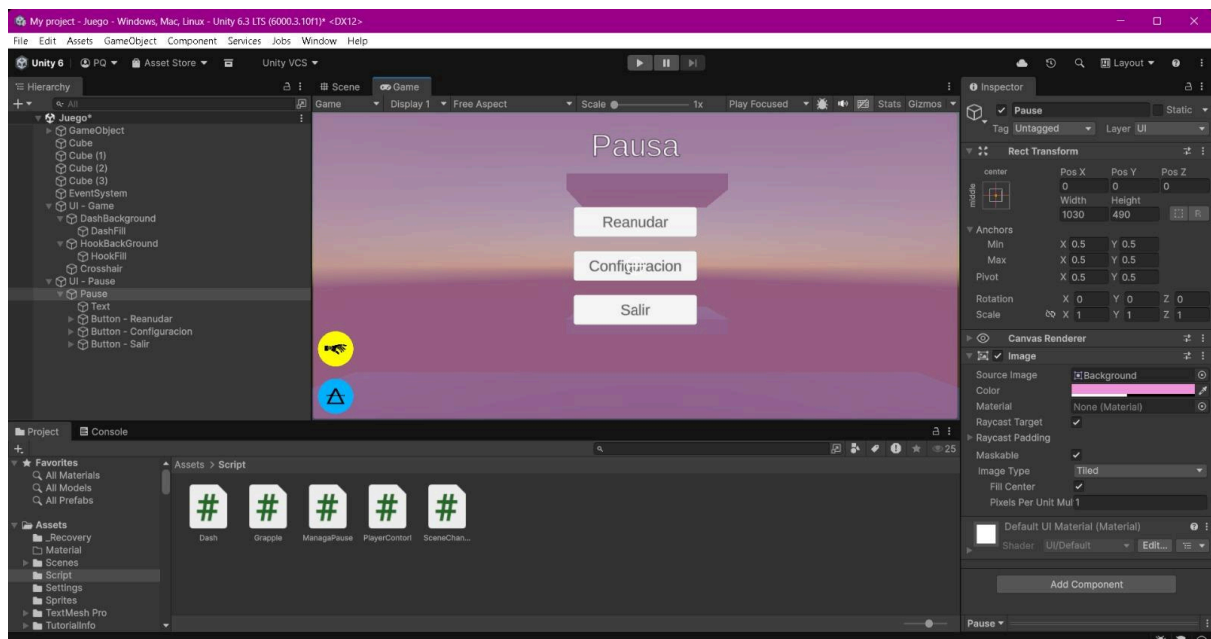
Documentación de nuestra tercera actualización: **Agregados CLAVE**

Descripción -Tercera actualización-

Tercera gran modificación del prototipo del juego en su versión Beta, estos son los agregados de la actualización:

- División de los códigos para una mejor modificación de los mismos.
- Creación de pantalla de inicio.

Agregado de pantalla de pausa: La gran pantalla de pausa



Agregamos la gran pantalla de pausa, la cual nos da múltiples opciones bastante apetitosas, **-Reanudar-**, **-Configuración-** y **-Salir-**.

Reanudar: Continuar con la partida en el momento que presionaste el botón de pausa (escape) para seguir con lo que dejaste en ese preciso momento.

Configuración: Abrir un panel capaz de mover los ajustes y mover los valores para la propia comodidad del jugador

Salir: Ejecutas la salida del juego, así, llevandote a la pantalla de inicio.

División de códigos: Mejora de organización

Código dividido: Dash

```
using System.Collections;
using UnityEngine;
using UnityEngine.UI;

[RequireComponent(typeof(CharacterController))]
public class PlayerDash : MonoBehaviour
{
    [Header("UI Dash")]
    public Image dashFillImage;

    [Header("Ajustes")]
    public float dashSpeed = 30f;
    public float dashDuration = 0.2f;
    public float dashCooldown = 1f;

    private float nextDashTime = 0f;
    private CharacterController characterController;
    private PlayerMovement playerMovement;

    void Start()
    {
        characterController = GetComponent<CharacterController>();
        playerMovement = GetComponent<PlayerMovement>();
    }

    void Update()
    {
        if (Time.timeScale == 0f) return;

        // Actualización de UI
        if (dashFillImage != null)
        {
            if (Time.time < nextDashTime)
                dashFillImage.fillAmount = 1f - ((nextDashTime - Time.time) / dashCooldown);
            else
                dashFillImage.fillAmount = 1f;
        }

        if (Input.GetKeyDown(KeyCode.Q) && Time.time >= nextDashTime)
        {
            StartCoroutine(DashRoutine());
        }
    }
}
```

```

IEnumerator DashRoutine()
{
    // 1. Apagamos el movimiento normal y la gravedad para que no interfieran
    if (playerMovement != null) playerMovement.enabled = false;

    nextDashTime = Time.time + dashCooldown;
    float startTime = Time.time;

    Vector3 dashDirection = new Vector3(Input.GetAxisRaw("Horizontal"), 0, Input.GetAxisRaw("Vertical"));
    dashDirection = transform.TransformDirection(dashDirection);

    if (dashDirection.sqrMagnitude == 0) dashDirection = transform.forward;

    Vector3 dashVelocity = dashDirection.normalized * dashSpeed;

    // 2. Ejecutamos el desplazamiento
    while (Time.time < startTime + dashDuration)
    {
        characterController.Move(dashVelocity * Time.deltaTime);
        yield return null;
    }

    // 3. Volvemos a encender el movimiento normal
    if (playerMovement != null) playerMovement.enabled = true;
}
}

```

Separamos el código de “Dash” para que este sea más fácil de controlar y también más cómodo a la hora de hacer un cambio dentro de él.

Código dividido: Grapple

```

using UnityEngine;
using UnityEngine.UI;

[RequireComponent(typeof(CharacterController))]
public class GrapplingHook : MonoBehaviour
{
    [Header("Referencias")]
    public Camera playerCamera;

    [Header("UI Gancho")]
    public Image grappleFillImage;
    public Image crosshairImage;
    public Color crosshairNormalColor = Color.white;
    public Color crosshairValidColor = Color.green;

    [Header("Ajustes Gancho")]
    public float maxDistance = 50f;
    public float grappleSpeed = 25f;
}

```

```

public KeyCode grappleKey = KeyCode.E;
public LayerMask grappleLayerMask = ~0;
public float grappleCooldown = 2f;

private float nextGrappleTime = 0f;
private bool isGrappling = false;
private Vector3 grapplePoint;

private CharacterController characterController;
private PlayerMovement playerMovement;

void Start()
{
    characterController = GetComponent<CharacterController>();
    playerMovement = GetComponent<PlayerMovement>();
}

void Update()
{
    if (Time.timeScale == 0f) return;

    UpdateGrappleUI();
    CheckCrosshairTarget();

    if (Input.GetKeyDown(grappleKey) && !isGrappling && Time.time >= nextGrappleTime)
    {
        StartGrapple();
    }

    if (isGrappling)
    {
        MoveToGrapplePoint();
    }
}

void UpdateGrappleUI()
{
    if (grappleFillImage != null)
    {
        if (Time.time < nextGrappleTime)
            grappleFillImage.fillAmount = 1f - ((nextGrappleTime - Time.time) / grappleCooldown);
        else
            grappleFillImage.fillAmount = 1f;
    }
}

void CheckCrosshairTarget()
{
    if (crosshairImage == null) return;

    if (isGrappling || Time.time < nextGrappleTime)
    {
        crosshairImage.color = crosshairNormalColor;
        return;
    }

    if (Physics.Raycast(playerCamera.transform.position, playerCamera.transform.forward, out __, maxDistance,
        grappleLayerMask))
    {
        crosshairImage.color = crosshairValidColor;
    }
    else
    {
        crosshairImage.color = crosshairNormalColor;
    }
}

```

```

void StartGrapple()
{
    if (Physics.Raycast(playerCamera.transform.position, playerCamera.transform.forward, out RaycastHit hit, maxDistance,
        grappleLayerMask))
    {
        grapplePoint = hit.point;
        isGrappling = true;

        // Apagamos el movimiento base
        if (playerMovement != null) playerMovement.enabled = false;
    }
}

void MoveToGrapplePoint()
{
    Vector3 directionToPoint = grapplePoint - transform.position;

    if (directionToPoint.sqrMagnitude < 4f)
    {
        isGrappling = false;
        nextGrappleTime = Time.time + grappleCooldown;

        // Llegamos al objetivo, encendemos el movimiento de nuevo
        if (playerMovement != null) playerMovement.enabled = true;
        return;
    }

    characterController.Move(directionToPoint.normalized * grappleSpeed * Time.unscaledDeltaTime);
}
}

```

Separamos el código de “Grappel” para que este sea más fácil de controlar y también más cómodo a la hora de hacer un cambio dentro de él.

Código dividido: **ManagePause**

```

using UnityEngine;
using UnityEngine.SceneManagement;

public class PauseManager : MonoBehaviour
{
    [Header("UI Sistema de Pausa")]
    public GameObject pauseMenuUI;
    public GameObject settingMenuUI;
    [Header("UI del Juego (HUD)")]
    public GameObject gameplayUI;

    private bool isPaused = false;

    void Start()

```

```

{
    if(settingMenuUI != null) settingMenuUI.SetActive(false);
    if (pauseMenuUI != null) pauseMenuUI.SetActive(false);
    if (gameplayUI != null) gameplayUI.SetActive(true);
}

void Update()
{
    if (Input.GetKeyDown(KeyCode.Escape))
    {
        if (isPaused) ResumeGame();
        else PauseGame();
    }
}

public void ResumeGame()
{
    isPaused = false;
    if (pauseMenuUI != null) pauseMenuUI.SetActive(false);
    if (settingMenuUI != null) settingMenuUI.SetActive(false);
    if (gameplayUI != null) gameplayUI.SetActive(true);

    Time.timeScale = 1f;

    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}

public void PauseGame()
{
    isPaused = true;
    if (pauseMenuUI != null) pauseMenuUI.SetActive(true);
    if (gameplayUI != null) gameplayUI.SetActive(false);

    Time.timeScale = 0f;

    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}

public void OpenSetting()
{
    if(pauseMenuUI != null) pauseMenuUI.SetActive(false);
    if(settingMenuUI != null) settingMenuUI.SetActive(true);
}

public void CloseSetting()
{
    if(pauseMenuUI != null) pauseMenuUI.SetActive(true);
    if(settingMenuUI != null) settingMenuUI.SetActive(false);
}

public void ChangeScene(string sceneName)
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(sceneName);
}
}

```

Separamos el código de “ManagePause” para que este sea más fácil de controlar y también más cómodo a la hora de hacer un cambio dentro de él.

Código dividido: PlayerControl

```
using UnityEngine;

[RequireComponent(typeof(CharacterController))]
public class PlayerMovement : MonoBehaviour
{
    [Header("Referencias")]
    public Camera playerCamera;
    private CharacterController characterController;

    [Header("Movimiento")]
    public float walkSpeed = 12f;
    public float runSpeed = 24f;
    public float jumpPower = 7f;
    public float gravity = 10f;

    private float baseWalkSpeed;
    private float baseRunSpeed;

    [Header("Camara")]
    public float lookSpeed = 2f;
    public float lookXLimit = 45f;
    private float rotationX = 0;

    [Header("Agacharse")]
    public float defaultHeight = 2f;
    public float crouchHeight = 1f;
    public float crouchSpeed = 3f;
    private bool isCrouching = false;

    private Vector3 moveDirection = Vector3.zero;

    void Start()
    {
        characterController = GetComponent<CharacterController>();
        baseWalkSpeed = walkSpeed;
        baseRunSpeed = runSpeed;

        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    void Update()
    {
        // Si el tiempo está congelado (Pausa), no hacemos nada
        if (Time.timeScale == 0f) return;

        Vector3 forward = transform.forward;
        Vector3 right = transform.right;

        bool isRunning = Input.GetKey(KeyCode.LeftShift);
        float currentSpeedMultiplier = isRunning ? runSpeed : walkSpeed;

        float inputVertical = Input.GetAxisRaw("Vertical");
        float inputHorizontal = Input.GetAxisRaw("Horizontal");
        float movementDirectionY = moveDirection.y;
```

```

Vector3 horizontalMove = (forward * inputVertical) + (right * inputHorizontal);

if (horizontalMove.sqrMagnitude == 0)
    moveDirection = Vector3.zero;
else
    moveDirection = horizontalMove.normalized * currentSpeedMultiplier;

if (Input.GetButton("Jump") && characterController.isGrounded)
    moveDirection.y = jumpPower;
else
    moveDirection.y = movementDirectionY;

if (!characterController.isGrounded)
    moveDirection.y -= gravity * Time.deltaTime;

// Agacharse
bool wantToCrouch = Input.GetKey(KeyCode.R);
if (wantToCrouch && !isCrouching)
{
    characterController.height = crouchHeight;
    walkSpeed = crouchSpeed;
    runSpeed = crouchSpeed;
    isCrouching = true;
}
else if (!wantToCrouch && isCrouching)
{
    characterController.height = defaultHeight;
    walkSpeed = baseWalkSpeed;
    runSpeed = baseRunSpeed;
    isCrouching = false;
}

characterController.Move(moveDirection * Time.deltaTime);

// Rotacion Camara
rotationX -= Input.GetAxis("Mouse Y") * lookSpeed;
rotationX = Mathf.Clamp(rotationX, -lookXLimit, lookXLimit);
playerCamera.transform.localRotation = Quaternion.Euler(rotationX, 0, 0);
transform.Rotate(Vector3.up, Input.GetAxis("Mouse X") * lookSpeed);
}
}

```

Separamos el código de “PlayerControl” para que este sea más fácil de controlar y también más cómodo a la hora de hacer un cambio dentro de él.